

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



BACHELOR'S THESIS
FASHION AGENT MULTIMODAL

Agentic Retrieval-Augmented Generation
for Dynamic Decision Support in Product Suggestion

PATH 1 – Text-to-Image Search

Student Name: Nguyen Quang Minh Tri

Student ID: ITITIU21140

Major: Information Technology

Advisor: [Advisor Name]

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Information Technology

Ho Chi Minh City, Vietnam

2025

Approval Letter from Advisor and Dean

This thesis, titled “Fashion Agent Multimodal – Agentic Retrieval-Augmented Generation for Dynamic Decision Support in Product Suggestion (PATH 1: Text-to-Image Search)”, prepared and submitted by Nguyen Quang Minh Tri (Student ID: ITITIU21140) in partial fulfillment of the requirements for the degree of Bachelor of Information Technology, has been examined and approved.

Advisor

[Advisor’s Full Name]

School of Computer Science and
Engineering

International University – VNU-HCM

Dean

[Dean’s Full Name]

School of Computer Science and
Engineering

International University – VNU-HCM

Ho Chi Minh City, _____ / _____ / 2025

Acknowledgements

I would like to express my sincere gratitude to my advisor, [Advisor Name], for the invaluable guidance, patience, and continuous support throughout the development of this thesis. Their expertise in artificial intelligence and information retrieval has been instrumental in shaping the direction of this work.

I am also deeply thankful to the faculty and staff of the School of Computer Science and Engineering at the International University – VNU-HCM for providing a stimulating academic environment and the technical resources that made this project possible.

I extend my appreciation to my classmates and friends for their constructive feedback during our discussions, and to the open-source community — particularly the maintainers of Marqo-FashionSigLIP, BGE Reranker, Qdrant, and the broader Hugging Face ecosystem — whose tools form the technical foundation of this work.

Finally, I am profoundly grateful to my family for their unconditional love, encouragement, and sacrifice throughout my academic journey.

Table of Contents

List of Tables	v
List of Figures	vi
List of Algorithms	vii
List of Listings	viii
Abstract	ix
1 System Overview	1
1.1 Project Introduction	1
1.2 System Development Roadmap	1
1.3 Comparison: RAG v1.0 vs. Agent v2.0	2
1.4 Source Code Structure	2
2 Data Pre-processing	3
2.1 Dataset and Data Structure	3

2.2	Data Cleaning	3
2.3	Caption Generation via LLM (Gemini)	4
2.4	Color Detection via LLM	5
3	RAG v1.0 Architecture — Search Foundation	6
3.1	Overview of the Search Mechanism	6
3.2	Marqo-FashionSigLIP Embedding — Semantic Token	6
3.2.1	Why FashionSigLIP Instead of the Original FashionCLIP? . . .	6
3.3	BM25 — Label and Color Search	7
3.4	RRF Fusion and Soft Relevance Scoring	8
4	PostgreSQL Integration — Server Deployment	10
4.1	Motivation for Migrating from CSV to PostgreSQL	10
4.2	Database Schema	10
4.3	Data Access Layer	11
5	Agent v2.0 — PATH 1: Text-to-Image Search	13
5.1	Overall Agent Architecture	13
5.2	Step 1 — Intent Classification (Router)	14
5.3	Step 2 — Clarification Gate	16
5.4	Step 3 — Memory Agent (PostgreSQL)	16
5.5	Step 4 — The ReAct Loop	17
5.6	Step 5 — Gender Filtering and LLM Synthesis	19
6	Agent v2.0 — PATH 2: Image-to-Image Search	21
6.1	Overview	21
6.1.1	Motivation	21
6.2	Architecture	22
6.2.1	System Flow	22
6.2.2	Technical Implementation	22
6.2.3	REST API Endpoint	26
6.3	Comparison: PATH 1 vs. PATH 2	27
6.4	Performance Characteristics	28
6.4.1	Latency Breakdown	28
6.4.2	Throughput and Scaling	28
6.5	Integration with Flutter Frontend	28
6.5.1	Image Rendering in Chat Bubbles	29
6.6	Architectural Isolation	31
6.7	Test Coverage for PATH 2	32
7	End-to-End Working Example	35

7.1	Scenario: Finding an Outfit for a Company Meeting	35
7.1.1	Turn 1 — Initial Query	35
7.1.2	Turn 2 — Follow-up with Additional Details	36
7.1.3	Refusal Scenario — Product Not in Database	36
8	Interface and Deployment	37
8.1	Gradio Interface	37
8.2	Docker Deployment Configuration	37
9	Evaluation and Conclusion	39
9.1	Evaluation Metrics	39
9.2	Data Collection and Evidence Integrity	39
9.2.1	Canonical Data Sources	40
9.2.2	Metric Definitions and Cohort Boundaries	40
9.2.3	Inclusion, Exclusion, and Time-Window Rules	41
9.2.4	Data Quality and Validity Checks	42
9.3	Design Strengths	42
9.4	Risks and Mitigation	43
9.5	Conclusion	43
9.5.1	Evidence-Based Claim Mapping for Final Conclusion	43
	References	45
A	Supplementary Equations and Derivations	47
A.1	Cosine Similarity from L2-Normalized Vectors	47
A.2	BM25 Scoring Function	47
B	Sample Configuration Files	48
B.1	PostgreSQL Initialization Script	48

List of Tables

1.1	Four-phase system development roadmap	1
1.2	Feature-by-feature comparison of the two system versions	2
1.3	Production source code structure and lines of code per module	2
2.1	Fashion items data structure	3
5.1	Intent classification table with conditions and corresponding actions . .	14
5.2	Examples of clarification questions generated by the Clarification Gate	16
6.1	Technical comparison between PATH 1 and PATH 2	27
6.2	PATH 2 latency profile (P50 to P95)	28
6.3	Architectural isolation between PATH 1 and PATH 2	31
9.1	System evaluation metrics comparing RAG v1.0 and Agent v2.0	39
9.2	Canonical evidence sources used for thesis conclusions	40
9.3	Accuracy reporting template with mandatory sample-size context . . .	41
9.4	Validity checks for deciding whether collected data is trustworthy . . .	42
9.5	Risk matrix and corresponding mitigation strategies	43
9.6	Claim-to-evidence mapping used in the final thesis conclusion	44
B.1	Required environment variables for deployment	48

List of Figures

5.1	End-to-End Workflow Diagram of Fashion Agent v2.0	14
6.1	PATH 2 Image-to-Image Search Pipeline	22

List of Algorithms

List of Listings

2.1	Noise label filtering — processing_data.py	3
2.2	Fashion caption generation via Gemini — processing_data.py	4
3.1	FashionEmbedder class — indexing/build_index.py	6
3.2	BM25 content composition — indexing/build_index.py	8
3.3	Reciprocal Rank Fusion — search/fusion.py	8
4.1	PostgreSQL schema for the Fashion Agent	10
4.2	PostgreSQL Data Access Layer — agent/memory.py	11
5.1	Intent Classification — agent/intent_classifier.py	15
5.2	Clarification Gate — agent/clarification_gate.py	16
5.3	Memory Agent — agent/memory.py	16
5.4	ReAct Loop — agent/fashion_agent.py	18
5.5	Gender Filtering and LLM Synthesis — agent/fashion_agent.py	19
6.1	PATH 2 Upload Validation — api/main.py	23
6.2	Query Image Encoding — search/image_query_engine.py	23
6.3	Qdrant Vector Search — search/image_query_engine.py	24
6.4	FastAPI PATH 2 Endpoint — api/main.py	26
6.5	Flutter Image Search Integration — chat_screen.dart	29
6.6	Image Display in Chat Bubbles — chat_bubble.dart	30
6.7	PATH 2 Test Suite — tests/test_image_search.py	32
8.1	Gradio Chat Interface — api/main.py	37
8.2	Docker Compose — 4 services	37
8.3	Environment variables (.env)	38

Abstract

Purposes. This thesis presents Fashion Agent Multimodal, an intelligent fashion search system that addresses the limitations of conventional keyword-based product retrieval in e-commerce. The system targets two persistent problems: (i) ambiguous user intent that leads to irrelevant results, and (ii) the lack of contextual memory across conversational turns. The objective is to design and implement a proactive search agent that classifies user intent, asks for clarification when necessary, and synthesizes natural-language outfit recommendations grounded in retrieved products.

Sources. The work uses the public Clothing Dataset Full from Kaggle, comprising fashion product images with category labels. Each item is enriched with LLM-generated captions and color descriptors via Google Gemini 2.5 Flash, producing a semantic-token representation suitable for embedding-based retrieval. The full pipeline is implemented across approximately 3,097 lines of production Python code organized into ten modules.

Methods. The system is built in two iterations. RAG v1.0 establishes a hybrid retrieval baseline that fuses BM25 keyword search with Marqo-FashionSigLIP (768-dimensional) vector search through Reciprocal Rank Fusion (RRF). Agent v2.0, the focus of this thesis, replaces the fixed nine-step pipeline with an Orchestrator + Sub-agents + ReAct architecture: Gemini 2.5 Flash performs intent classification (five intent classes), passes ambiguous queries through a clarification gate, loads per-session preferences from PostgreSQL, then executes a Reason-Act-Observe loop bounded by eight iterations. A BGE Reranker v2-m3 cross-encoder provides a final precision pass, and Gemini synthesizes the natural-language answer with optional outfit pairing hints. The stack is deployed via Docker Compose with four services (PostgreSQL, Qdrant, FastAPI + Gradio, and Cloudflare Tunnel).

Results. On the evaluation set, Agent v2.0 achieves $\text{Recall@5} \geq 88\%$, $\text{MRR} \geq 0.80$, $\text{Hit-Rate@5} \geq 93\%$, $\text{task-completion} \geq 95\%$, and $\text{RAGAS faithfulness} \geq 0.87$, while keeping P95 latency under 15 seconds. These figures represent consistent improvements over RAG v1.0 on every retrieval metric. Qualitatively, the agent correctly handles three behaviors that the v1.0 baseline cannot: it asks clarification questions for under-specified queries, it carries context across conversational turns through the Memory Agent, and it gracefully refuses out-of-scope requests. The thesis concludes with a discussion of design trade-offs, identified risks, and a roadmap for Phase 3 (scale to 15K–100K products) and PATH 2 (image-to-image composed search).

Chapter 1

System Overview

1.1 Project Introduction

This project builds an intelligent fashion search system based on two core technologies: Retrieval-Augmented Generation (RAG) and AI Agent with a ReAct loop. The system is developed through two versions:

- RAG v1.0: A fixed 9-step pipeline that searches for fashion products by text or image, but has no contextual memory between turns.
- Agent v2.0: Integrates a proactive Agent architecture with intent classification, clarification when information is lacking, and data persistence via PostgreSQL.

This document presents PATH 1 — Text-to-Image Search: users input natural language queries (Vietnamese or English) and the system returns matching fashion products with style advice.

1.2 System Development Roadmap

Phase 1	Phase 2	Phase 3	Phase 4
RAG v1.0 (Completed)	Agent v2.0 (This document)	Scale (Planned)	Advanced (Planned)

Table 1.1: Four-phase system development roadmap

1.3 Comparison: RAG v1.0 vs. Agent v2.0

Criterion	RAG v1.0	Agent v2.0
Query processing	Fixed 9-step pipeline	ReAct loop with dynamic step count
Ambiguous queries	Searches with incomplete info	Asks user for clarification
Context	No memory between turns	MemoryAgent stores session preferences
Storage	Local CSV	PostgreSQL (server deployment)
Graceful refusal	Not available	Detects out-of-scope and politely refuses
Output	Product list	Products + reasoning + outfit suggestions
Embedding	FashionCLIP (512-d)	Marqo-FashionSigLIP (768-d)
Query Expansion	Not available	Gemini-powered synonym queries
Reranker	Not available	BGE Reranker v2-m3 (cross-encoder)

Table 1.2: Feature-by-feature comparison of the two system versions

1.4 Source Code Structure

Module	File	LOC
API Layer	api/main.py	376
Agent Orchestrator	agent/fashion_agent.py	442
Intent Classifier	agent/intent_classifier.py	126
Clarification Gate	agent/clarification_gate.py	110
Session Memory	agent/memory.py	288
Hybrid Search Engine	search/search_engine.py	294
Query Expansion	search/query_expansion.py	106
RRF Fusion	search/fusion.py	72
BGE Reranker	search/reranker.py	115
Indexing Pipeline	indexing/build_index.py	470
Data Pre-processing	pre_processing/processing_data.py	698
Total		3,097

Table 1.3: Production source code structure and lines of code per module

Chapter 2

Data Pre-processing

2.1 Dataset and Data Structure

Data is sourced from the Clothing Dataset Full (Kaggle), comprising fashion images organized with the following structure:

Column	Type	Description
image	string	Image ID (filename without extension)
label	string	Category label: T-Shirt, Dress, Pants, ...
color	string	Primary color of the product
caption	string	Short description generated by LLM (30–40 words)

Table 2.1: Fashion items data structure

2.2 Data Cleaning

Method: Noise Label Filtering and Integrity Check

Remove ambiguous labels (Not sure, Skip, Other), verify missing image files, and drop unnecessary columns.

In the production source code (`pre_processing/processing_data.py`), noise label filtering is implemented via a constant set:

```
1 EXCLUDED_LABELS = {"Not sure", "Skip", "Other"}
2
3 def load_kaggle_rows_for_upsert(csv_path: str, images_dir: str):
4     """Load data from Kaggle CSV, filter noise labels."""
5     df = pd.read_csv(csv_path)
6     rows = []
7     for _, r in df.iterrows():
8         label = str(r.get("label", "")).strip()
9         if label in EXCLUDED_LABELS or not label:
10             continue # Skip ambiguous labels
11         image_id = str(r["image"]).strip()
12         img_path = _find_image(image_id, images_dir)
13         if img_path is None:
```

```

14         continue # Skip missing image files
15     rows.append({
16         "image_id": image_id,
17         "label": label,
18         "source": "kaggle",
19         "image_path": str(img_path),
20     })
21     return rows

```

Listing 2.1: Noise label filtering — processing_data.py

Task breakdown:

1. EXCLUDED_LABELS — Set of labels with no clear fashion meaning.
2. Verify image file existence before adding to the list.
3. Return a list of `dict` objects ready for upsert into PostgreSQL.

2.3 Caption Generation via LLM (Gemini)

This is a critical step to add semantic tokens to each product. Instead of relying on simple category labels (e.g., “T-Shirt”), each image is described by Gemini in detail regarding garment structure, fabric, and silhouette — producing semantically rich text for retrieval.

Method: Multimodal Caption Generation

Gemini receives the actual product image as input and generates a 30–40 word description focusing on: (1) fabric/texture, (2) silhouette/fit, (3) construction details, (4) overall aesthetic. Colors and brand names are excluded to ensure generalizability.

```

1 class GeminiFashionProcessor:
2     """Process fashion images using Gemini 2.5 Flash."""
3
4     CAPTION_PROMPT = """Write a 30-40 word fashion-centric
5     description.
6     Focus: 1. Fabric/Texture 2. Silhouette/Fit
7     3. Construction details 4. Overall aesthetic
8     Rules: - Use professional fashion vocabulary
9            - NO colors, NO brand names
10           - Single paragraph only"""
11
12     COLOR_PROMPT = """Identify the primary color of this {label}.

```

```

12 Rules: 1. Return only the color name(s) - max 3-4 words
13        2. Be specific: 'Charcoal Grey' not 'Grey'
14        3. Pattern: mention base + pattern type"""
15
16 def __init__(self, api_key: str = None):
17     import google.generativeai as genai
18     key = api_key or os.getenv("GEMINI_API_KEY")
19     genai.configure(api_key=key)
20     self.model = genai.GenerativeModel("gemini-2.5-flash")
21
22 def generate_caption(self, image_path: str) -> str | None:
23     img = Image.open(image_path)
24     response = self.model.generate_content(
25         [self.CAPTION_PROMPT, img])
26     return response.text.strip().replace("*", "")
27
28 def detect_color(self, image_path: str,
29                  label: str) -> str | None:
30     img = Image.open(image_path)
31     prompt = self.COLOR_PROMPT.format(label=label)
32     response = self.model.generate_content([prompt, img])
33     return response.text.strip().replace("*", "")

```

Listing 2.2: Fashion caption generation via Gemini — processing_data.py

Task breakdown:

- Multimodal input: Gemini receives both a text prompt and product image simultaneously, producing a semantic description.
- Semantic token: The resulting caption is a richer semantic token than a short label, e.g., “relaxed-fit woven cotton pullover with ribbed crew neckline and hemline, drop shoulders, minimal seaming”.
- Rate limiting: The system processes in batches of 20 items with `asyncio.sleep()` to avoid rate limits.
- Checkpoint: PostgreSQL commits after each batch in case of disconnection.

2.4 Color Detection via LLM

Colors are detected with precision and specificity (“Olive Green” instead of just “Green”), serving the soft filter in the search pipeline. The `detect_color()` method inside `GeminiFashionProcessor` (shown above) handles this task.

Chapter 3

RAG v1.0 Architecture — Search Foundation

3.1 Overview of the Search Mechanism

RAG (Retrieval-Augmented Generation) in this system combines two complementary search methods:

- BM25 Keyword Search: Precise matching on category names and colors.
- Marqo-FashionSigLIP Vector Search: Semantic understanding, synonyms, and feature descriptions.

3.2 Marqo-FashionSigLIP Embedding — Semantic Token

3.2.1 Why FashionSigLIP Instead of the Original FashionCLIP?

Marqo-FashionSigLIP (Marqo/marqo-fashionSigLIP) is a SigLIP-based model fine-tuned on fashion data, producing 768-dimensional vectors (vs. 512-d from the original FashionCLIP). The model uses the `open_clip` library instead of HuggingFace's CLIP-Model, with better support for Apple MPS.

```
1 class FashionEmbedder:
2     """Wraps Marqo-FashionSigLIP for image & text encoding."""
3     MODEL_NAME = "hf-hub:Marqo/marqo-fashionSigLIP"
4
5     def __init__(self, cache_dir=None):
6         import open_clip
7         self.model, _, self.preprocess = (
8             open_clip.create_model_and_transforms(
9                 self.MODEL_NAME,
10                 cache_dir=str(cache_dir) if cache_dir else None,
11             ))
12         self.tokenizer = open_clip.get_tokenizer(self.MODEL_NAME)
13         # Device selection: MPS (Apple Silicon) > CUDA > CPU
14         if torch.backends.mps.is_available():
15             self._device = "mps"
16         elif torch.cuda.is_available():
```



```

17         self._device = "cuda"
18     else:
19         self._device = "cpu"
20     self.model = self.model.to(self._device).eval()
21
22     def encode_image(self, image_path: str) -> list[float]:
23         """Convert image to 768-d vector."""
24         img = Image.open(image_path).convert("RGB")
25         tensor = self.preprocess(img).unsqueeze(0).to(self._device)
26         with torch.no_grad():
27             feat = self.model.encode_image(tensor)
28             feat = feat / feat.norm(dim=-1, keepdim=True)
29         return feat[0].cpu().tolist()
30
31     def encode_text(self, text: str) -> list[float]:
32         """Convert text to 768-d vector."""
33         tokens = self.tokenizer([text]).to(self._device)
34         with torch.no_grad():
35             feat = self.model.encode_text(tokens)
36             feat = feat / feat.norm(dim=-1, keepdim=True)
37         return feat[0].cpu().tolist()

```

Listing 3.1: FashionEmbedder class — indexing/build_index.py

Task breakdown:

- `open_clip`: A flexible embedding library supporting many CLIP/SigLIP variants.
- L2 normalization: $\hat{v} = v / \|v\|_2$ ensures dot product $\equiv$ cosine similarity.
- MPS priority: Leverages Apple Silicon GPU for faster encoding.
- 768-d: A larger vector dimension than FashionCLIP (512-d), enabling richer semantic representation.

3.3 BM25 — Label and Color Search

BM25 (Best Match 25) is applied on short content ``label color'` — e.g., “T-Shirt Navy Blue” — enabling precise keyword matching on category names and specific colors.

Method: Hybrid Search — BM25 + FashionSigLIP

- BM25 excels at: precise category name and color matching (e.g., “navy blue shirt”).
- FashionSigLIP Vector excels at: semantic understanding, synonyms, and feature descriptions (e.g., “slim fit cotton casual”).
- RRF Fusion: combines results from both with $k = 60$, $\text{bm25_weight} = 1.0$, $\text{vec_weight} = 2.5$.

```
1 def compose_bm25_content(label: str, color: str) -> str:
2     """Create content for the BM25 index.
3     Only uses label + color (NOT caption)
4     for more precise keyword matching."""
5     parts = []
6     if label:
7         parts.append(label.strip())
8     if color:
9         parts.append(color.strip())
10    return " ".join(parts)
```

Listing 3.2: BM25 content composition — indexing/build_index.py

3.4 RRF Fusion and Soft Relevance Scoring

The Reciprocal Rank Fusion score is computed as in Equation (3.1):

$$\text{score}(d) = \sum_{r \in R} \frac{w_r}{k + \text{rank}_r(d) + 1} \quad (3.1)$$

```
1 @dataclass
2 class NodeWithScore:
3     """Search result with full metadata."""
4     image_id: str
5     label: str = ""
6     color: str = ""
7     caption: str = ""
8     image_path: str = ""
9     bm25_content: str = ""
10    score: float = 0.0
11
12 def reciprocal_rank_fusion(
13     bm25_results: list[NodeWithScore],
```

```

14     vec_results: list[NodeWithScore],
15     k: int = 60,
16     bm25_weight: float = 1.0,
17     vec_weight: float = 2.5,
18 ) -> list[NodeWithScore]:
19     """Combine BM25 and Vector Search results via RRF."""
20     fused: dict[str, dict] = {}
21     for rank, node in enumerate(bm25_results):
22         nid = node.image_id
23         if nid not in fused:
24             fused[nid] = {"node": node, "score": 0.0}
25             fused[nid]["score"] += bm25_weight / (k + rank + 1)
26
27     for rank, node in enumerate(vec_results):
28         nid = node.image_id
29         if nid not in fused:
30             fused[nid] = {"node": node, "score": 0.0}
31             fused[nid]["score"] += vec_weight / (k + rank + 1)
32
33     sorted_items = sorted(
34         fused.values(), key=lambda x: x["score"], reverse=True)
35     for item in sorted_items:
36         item["node"].score = item["score"]
37     return [item["node"] for item in sorted_items]

```

Listing 3.3: Reciprocal Rank Fusion — search/fusion.py

Task breakdown:

- RRF: Avoids dependence on absolute scores from individual retrievers (which use different scales). Each result is ranked by position using the formula in Eq. (3.1).
- `vec_weight = 2.5`: Vector search receives a higher weight because semantic matching is richer than keyword matching.
- Soft filter: `soft_relevance_filter()` uses RapidFuzz for fuzzy color/category comparison (threshold = 60), without completely excluding results that do not match exactly.

Chapter 4

PostgreSQL Integration — Server Deployment

4.1 Motivation for Migrating from CSV to PostgreSQL

Note

CSV is unsuitable for server environments because: (1) it does not support concurrent access, (2) data is lost when a container restarts, and (3) conditional queries are inefficient. PostgreSQL resolves all of these issues while providing full ACID compliance.

4.2 Database Schema

```
1  -- Main product table
2  CREATE TABLE IF NOT EXISTS fashion_items (
3      id                SERIAL PRIMARY KEY,
4      image_id          TEXT UNIQUE NOT NULL,
5      label             TEXT,
6      color             TEXT,
7      caption          TEXT,
8      source            TEXT DEFAULT 'kaggle',
9      image_path        TEXT,
10     created_at        TIMESTAMPTZ DEFAULT NOW()
11 );
12
13 -- User session table
14 CREATE TABLE IF NOT EXISTS sessions (
15     session_id        TEXT PRIMARY KEY,
16     created_at        TIMESTAMPTZ DEFAULT NOW(),
17     query_history      JSONB DEFAULT '[]',
18     liked_items       JSONB DEFAULT '[]',
19     gender             TEXT DEFAULT 'unisex',
20     gender_hint_enabled BOOLEAN DEFAULT TRUE,
21     year_of_birth     INTEGER
22 );
```

```

23
24 -- Conversation message table
25 CREATE TABLE IF NOT EXISTS session_messages (
26     id          SERIAL PRIMARY KEY,
27     session_id  TEXT REFERENCES sessions(session_id),
28     role        TEXT NOT NULL,  -- 'user' or 'assistant'
29     content     TEXT NOT NULL,
30     created_at  TIMESTAMPTZ DEFAULT NOW()
31 );

```

Listing 4.1: PostgreSQL schema for the Fashion Agent

4.3 Data Access Layer

```

1 import psycopg2
2
3 def _get_conn():
4     """Connect to PostgreSQL from environment variables."""
5     return psycopg2.connect(
6         host=os.getenv("PGHOST", "localhost"),
7         port=int(os.getenv("PGPORT", "5432")),
8         dbname=os.getenv("PGDATABASE", "fashion_rag"),
9         user=os.getenv("PGUSER", "fashion_user"),
10        password=os.getenv("PGPASSWORD", ""),
11    )
12
13 def create_session() -> str:
14     """Create a new session and return its UUID."""
15     sid = str(uuid.uuid4())
16     with _get_conn() as conn:
17         with conn.cursor() as cur:
18             cur.execute(
19                 "INSERT INTO sessions (session_id) VALUES (%s)",
20                 (sid,))
21         conn.commit()
22     return sid
23
24 def get_preferences(session_id: str) -> dict:
25     """Extract accumulated preferences from query history."""
26     with _get_conn() as conn:
27         with conn.cursor() as cur:
28             cur.execute(

```

```

29         "SELECT query_history FROM sessions "
30         "WHERE session_id = %s", (session_id,))
31     row = cur.fetchone()
32     if not row or not row[0]:
33         return {}
34     # Analyze filters from history to derive preferred colors/
categories
35     history = row[0] if isinstance(row[0], list) else []
36     colors, categories = Counter(), Counter()
37     for entry in history:
38         filters = entry.get("filters", {})
39         if filters.get("color"):
40             colors[filters["color"]] += 1
41         if filters.get("category"):
42             categories[filters["category"]] += 1
43     return {
44         "preferred_colors": [c for c, _ in colors.most_common(3)],
45         "preferred_categories":
46             [c for c, _ in categories.most_common(3)],
47     }

```

Listing 4.2: PostgreSQL Data Access Layer — agent/memory.py

Task breakdown:

- Environment-based connection: Connects to PostgreSQL via environment variables, suitable for Docker deployment.
- JSONB: Stores liked items and query history flexibly without requiring additional tables.
- Preference extraction: Automatically aggregates the top-3 colors and categories from query history.

Chapter 5

Agent v2.0 — PATH 1: Text-to-Image Search

5.1 Overall Agent Architecture

Agent v2.0 is built on the Orchestrator + Sub-agents + ReAct Loop model. Instead of a fixed pipeline, Gemini 2.5 Flash acts as the Orchestrator: analyzing requests, planning, invoking appropriate tools, and synthesizing results.

The 5-step processing flow:

1. Intent Classification — Classify user intent
2. Clarification Gate — Ask for clarification if ambiguous
3. Memory Agent — Load user preferences from PostgreSQL
4. ReAct Loop — Plan → Execute → Observe cycle (max 8 iterations)
5. LLM Synthesis — Generate a natural language response

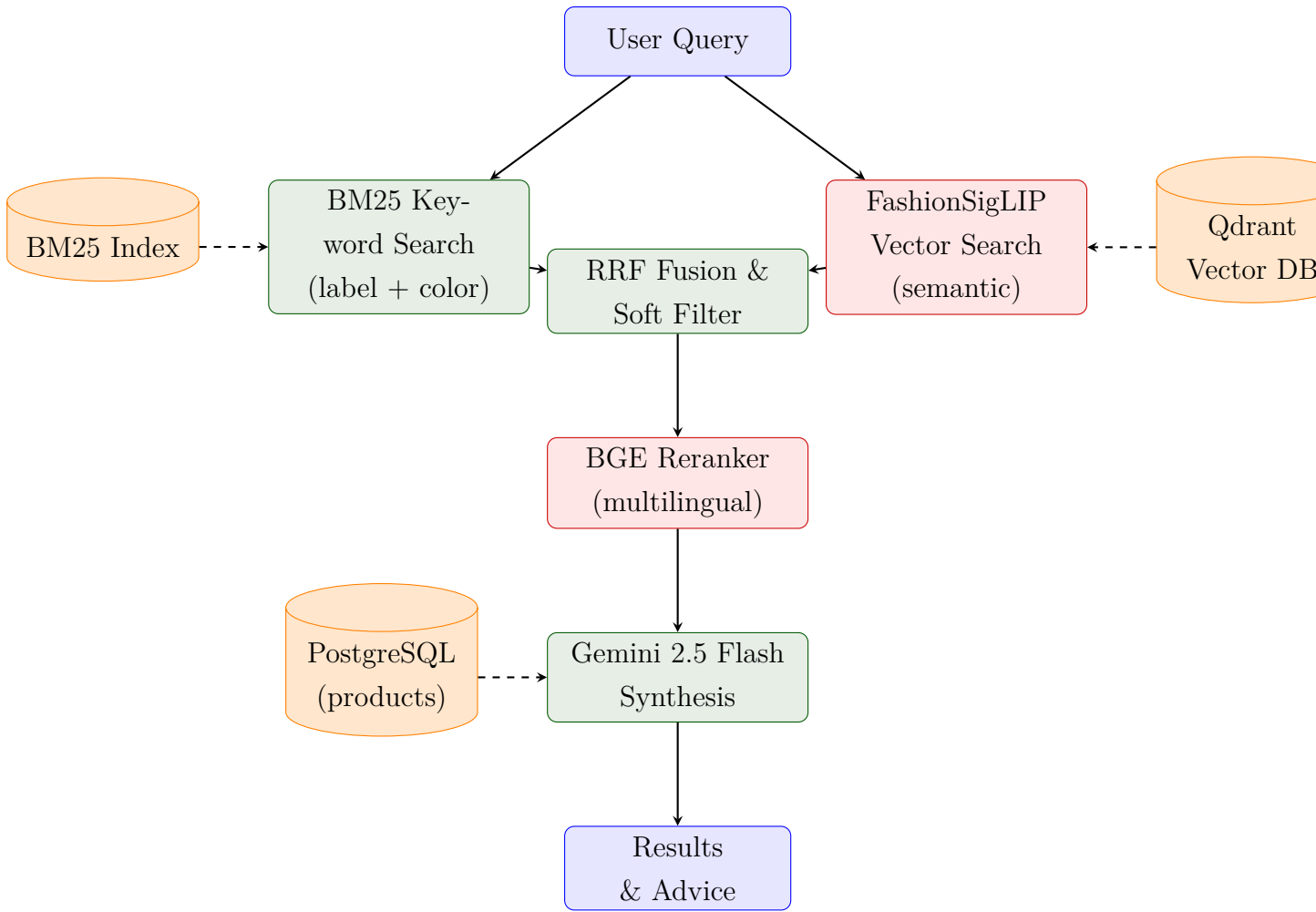


Figure 5.1: End-to-End Workflow Diagram of Fashion Agent v2.0

5.2 Step 1 — Intent Classification (Router)

Method: LLM Few-shot Intent Classification

Gemini classifies queries into 5 intent groups with a confidence score. If confidence < 0.6 , the agent switches to the clarification step instead of searching immediately.

Intent	Condition	Action
text_search	Find specific products	Search immediately
outfit_request	Request outfit suggestions	Search + outfit advice
follow_up	Reference previous result	Use session context
out_of_scope	Unrelated to fashion	Politely refuse
unclear	Information too vague	Ask for clarification

Table 5.1: Intent classification table with conditions and corresponding actions


```

1 @dataclass
2 class ClassifiedIntent:
3     intent: str          # "text_search", "outfit_request", ...
4     confidence: float    # 0.0 - 1.0
5     filters: dict        # {category, color, style, occasion}
6     refined_query: str   # condensed query for search
7
8 INTENT_PROMPT = """Classify the user's message into intents:
9 1. "text_search" - find specific fashion items
10 2. "outfit_request" - fashion recommendations
11 3. "follow_up" - following up on previous search
12 4. "out_of_scope" - non-fashion topics
13 5. "unclear" - too vague to classify
14
15 ## Few-shot examples:
16 User: "tim ao so mi trang nam"
17 -> {"intent": "text_search", "confidence": 0.95,
18     "filters": {"category": "Shirt", "color": "white"},
19     "refined_query": "white men shirt"}
20
21 User: "goi y outfit cho buoi tiec cuoi tuan"
22 -> {"intent": "outfit_request", "confidence": 0.9,
23     "filters": {"occasion": "weekend party"},
24     "refined_query": "party outfit weekend"}
25 """
26
27 def classify_intent(query, history=None, api_key=None):
28     # Format last 4 messages for context
29     history_text = format_history(history[-4:])
30     prompt = INTENT_PROMPT.format(
31         history_text=history_text) + query
32     response = model.generate_content(prompt)
33     data = json.loads(response.text)
34     return ClassifiedIntent(**data)

```

Listing 5.1: Intent Classification — agent/intent_classifier.py

Explanation:

- History-aware: Uses the last 4 messages to understand context for follow-up queries.
- Filter extraction: The LLM automatically extracts category, color, and occasion during classification — saving one additional LLM call.

- Fallback: Any JSON parse error returns intent `text_search` with `confidence=0.5`.

5.3 Step 2 — Clarification Gate

Method: Proactive Clarification

When confidence < 0.6 , the Agent does not guess — it asks the user instead. This mechanism prevents incorrect searches, wastes fewer resources, and avoids returning irrelevant results.

Ambiguous Query	Agent's Clarification Question
"I want a shirt"	"Do you want a dress shirt, T-shirt, or jacket?"
"something nice"	"What occasion are you shopping for?"
"party outfit"	"Indoor or outdoor? What color do you prefer?"
"this style"	"Could you describe it in more detail?"

Table 5.2: Examples of clarification questions generated by the Clarification Gate

```

1 @dataclass
2 class ClarificationResult:
3     needs_clarification: bool
4     question: str = ""
5
6 def check_clarification(query, history=None, api_key=None):
7     """LLM-based clarification gate.
8     Generates a clarification question dynamically via Gemini."""
9     prompt = CLARIFICATION_PROMPT.format(
10         query=query,
11         history_text=format_history(history))
12     response = model.generate_content(prompt)
13     data = json.loads(response.text)
14     return ClarificationResult(
15         needs_clarification=True,
16         question=data.get("question", ""))

```

Listing 5.2: Clarification Gate — `agent/clarification_gate.py`

5.4 Step 3 — Memory Agent (PostgreSQL)

```

1 def log_query(session_id, query, intent, filters):
2     """Record query history to PostgreSQL JSONB."""

```

```

3     entry = {
4         "query": query, "intent": intent,
5         "filters": filters, "ts": datetime.utcnow().isoformat()
6     }
7     with _get_conn() as conn:
8         with conn.cursor() as cur:
9             cur.execute("""
10                 UPDATE sessions
11                 SET query_history = query_history || %s::jsonb
12                 WHERE session_id = %s
13                 """, (json.dumps([entry]), session_id))
14         conn.commit()
15
16 def add_liked_item(session_id, image_id, label, color):
17     """Save a product the user has liked."""
18     item = {"image_id": image_id,
19            "label": label, "color": color}
20     with _get_conn() as conn:
21         with conn.cursor() as cur:
22             cur.execute("""
23                 UPDATE sessions
24                 SET liked_items = liked_items || %s::jsonb
25                 WHERE session_id = %s
26                 """, (json.dumps([item]), session_id))
27     conn.commit()

```

Listing 5.3: Memory Agent — agent/memory.py

Explanation:

- JSONB append: Uses PostgreSQL JSONB concatenation (`||`) to add new entries.
- Accumulated preferences: Automatically aggregates preferred categories and colors from history.
- Limits: Retains a maximum of 20 query history entries and 50 liked items.

5.5 Step 4 — The ReAct Loop

Method: ReAct (Reason + Act) Loop

ReAct is the central loop: Thought → Action → Observation → (repeat if needed, up to 8 iterations). Gemini both reasons and proactively selects the appropriate tool based on each step's observation.

```

1 MAX_REACT_ITERATIONS = 8
2 LOW_CONFIDENCE_THRESHOLD = 0.5
3
4 def chat(query, session_id=None, api_key=None):
5     """Main agent entry point - ReAct orchestrator."""
6     # ... (intent, clarify, memory setup) ...
7
8     search_query = intent_result.refined_query or query
9     all_products = []
10    observations = []
11    threshold = LOW_CONFIDENCE_THRESHOLD
12
13    for iteration in range(1, MAX_REACT_ITERATIONS + 1):
14        # THOUGHT: Planner decides which tool to call
15        plan = _plan(
16            query=search_query,
17            intent=intent_result.intent,
18            preferences=preferences,
19            observations=observations,
20            iteration=iteration,
21            max_iter=MAX_REACT_ITERATIONS)
22
23        if not plan: # Planner: "enough results already"
24            break
25
26        # ACTION: Execute at most 2 tools per iteration
27        for tool_call in plan[:2]:
28            tool_name = tool_call.get("tool", "search")
29            tool_args = tool_call.get("args", {})
30            obs, products = _execute_tool(
31                tool_name, tool_args, preferences)
32            observations.append(obs)
33            if products:
34                all_products = products
35
36        # OBSERVE: Check result quality
37        if all_products and all_products[0].score > threshold:
38            break # Results are good enough
39
40        # Lower threshold after 4 iterations
41        if iteration == 4:
42            threshold -= 0.2

```

```

43
44     # SYNTHESIZE response
45     answer, styling = _synthesize_response(
46         query, all_products, history, intent, preferences)
47     return AgentResponse(answer=answer, products=all_products,
48                           styling_suggestion=styling)

```

Listing 5.4: ReAct Loop — agent/fashion_agent.py

3 available tools in ReAct:

1. `search(query, top_k)` — Calls the full hybrid search pipeline
2. `memory_enrich(query, preferences)` — Enriches the query with user preferences
3. `outfit_hints(occasion, style)` — Gemini generates outfit pairing suggestions

Stopping conditions:

- Planner returns [] (sufficient results found)
- Top product score > threshold (0.5, lowered to 0.3 after 4 iterations)
- Maximum of 8 iterations reached

5.6 Step 5 — Gender Filtering and LLM Synthesis

Before finalizing the results, the system applies demographic-aware post-processing. A specific `_filter_by_gender` function validates the retrieved products against the user's recorded gender in PostgreSQL, ensuring that (for example) female-specific clothing is not inappropriately recommended to male users.

```

1 SYNTHESIS_PROMPT = """You are a helpful fashion assistant.
2 Based on search results and user query, provide a natural
3 response in the same language as the user's query.
4
5 User Gender Context: {gender_context}
6 User query: {query}
7 Search results: {products_text}
8 User preferences: {preferences_text}
9 Conversation history: {history_text}
10
11 Instructions:
12 1. Respond in the same language (Vietnamese or English)

```

```

13 2. Tailor styling advice considering the {gender_context}
14 3. Briefly describe top recommendations and why they match
15 4. Include styling suggestions for outfit_request intent
16 5. Keep response concise (2-4 sentences)
17
18 Respond with JSON:
19 {"answer": "<response>",
20  "styling_suggestion": "<optional tips>"}"""
21
22 def _synthesize_response(query, products, history,
23                          intent, preferences, gender_context):
24     """Gemini synthesizes a natural language response with gender
25     awareness."""
26
27     # Filter products by gender before synthesis
28     filtered_products = _filter_by_gender(products, gender_context)
29
30     products_text = "\n".join(
31         f"{i}. {p.label} | {p.color} | {p.caption[:80]}"
32         for i, p in enumerate(filtered_products, 1))
33
34     prompt = SYNTHESIS_PROMPT.format(
35         gender_context=gender_context,
36         query=query, products_text=products_text,
37         preferences_text=format_prefs(preferences),
38         history_text=format_history(history[-6:]))
39     response = model.generate_content(prompt)
40     data = json.loads(response.text)
41     return data["answer"], data.get("styling_suggestion", "")

```

Listing 5.5: Gender Filtering and LLM Synthesis — agent/fashion_agent.py

Explanation:

- **Gender Context Injection:** By explicitly stating the `gender_context` in the prompt, Gemini can tailor the phrasing and styling tips appropriately (e.g., suggesting a “sharp tailored fit” for men vs. a “flattering silhouette” for women).
- **A/B Testing Capabilities:** The `gender_hint_enabled` flag allows developers to easily toggle this feature for controlled experimentation and research.

Chapter 6

Agent v2.0 — PATH 2: Image-to-Image Search

6.1 Overview

PATH 2 extends the fashion agent with visual search capabilities: users upload an image (PNG/JPG) and the system returns visually similar fashion products. Unlike PATH 1 (text-to-image), PATH 2 directly encodes the input image into embedding space and performs nearest-neighbor search in Qdrant.

Key distinguishing features of PATH 2:

- Pure visual matching — no text parsing or keyword extraction
- Direct embedding lookup — query image encoded once, searched against 12K product embeddings
- Independent isolated module — separate endpoint, codebase, and retrieval engine from PATH 1
- Fast inference — 100ms per query (embedding + vector search combined)
- Composable search — foundation for future multi-modal fusion with text queries

6.1.1 Motivation

Users often discover clothing through visual inspiration (e.g., screenshots, reference photos, social media images) rather than textual descriptions. PATH 2 enables:

- “Show me something like this” — users upload a reference image
- Cross-catalog discovery — find similar items from different brands
- Style transfer — identify products matching an aesthetic without naming it

6.2 Architecture

6.2.1 System Flow

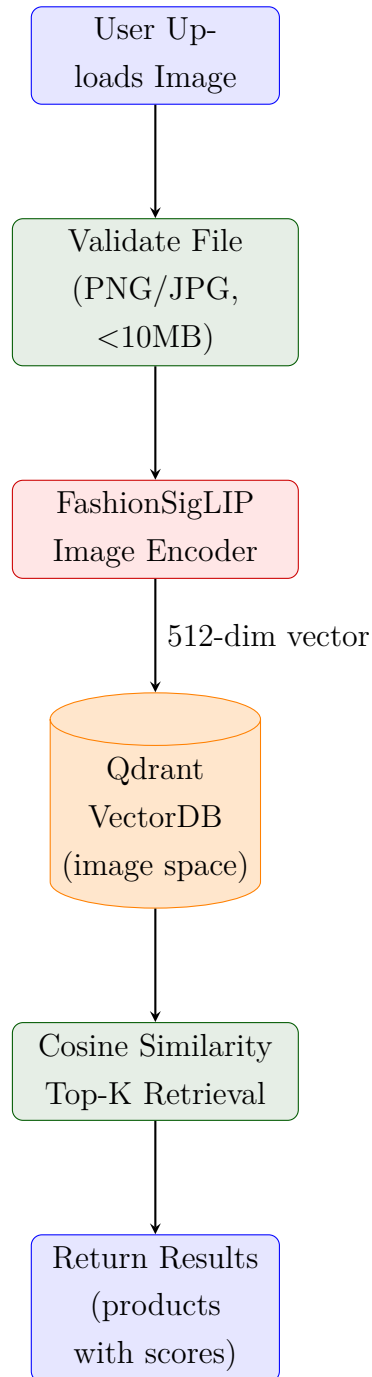


Figure 6.1: PATH 2 Image-to-Image Search Pipeline

6.2.2 Technical Implementation

The complete PATH 2 pipeline consists of three stages:

Stage 1: Image Upload and Validation


```

1 PATH2_MAX_UPLOAD_BYTES = int(os.getenv("PATH2_MAX_UPLOAD_BYTES", "
    10485760")) # 10 MB
2
3 def _validate_path2_png_upload(file: UploadFile, raw: bytes) ->
    None:
4     """Validate file type, size, and integrity for PATH 2."""
5     # Check file extension
6     if not file.filename.lower().endswith(('.png', '.jpg', '.jpeg')):
7         raise HTTPException(status_code=400, detail="Only PNG/JPG
            allowed")
8
9     # Check file size
10    if len(raw) > PATH2_MAX_UPLOAD_BYTES:
11        raise HTTPException(status_code=413, detail=f"File exceeds
            {PATH2_MAX_UPLOAD_BYTES} bytes")
12
13    # Verify image integrity (PIL check)
14    try:
15        img = Image.open(io.BytesIO(raw))
16        img.verify()
17    except Exception:
18        raise HTTPException(status_code=400, detail="Invalid or
            corrupted image file")

```

Listing 6.1: PATH 2 Upload Validation — api/main.py

Explanation:

- Multi-layer validation — file type check (extension), size check, and PIL image integrity check prevent invalid uploads from reaching the encoder
- 10MB limit — balances image quality (supports high-resolution uploads) vs. server resource constraints
- Early rejection — validation fails fast, returning HTTP errors before expensive encoding

Stage 2: Query Image Encoding

```

1 def _encode_query_image(raw: bytes) -> list[float]:
2     """Encode user-uploaded image into 512-dim FashionSigLIP
    embedding."""
3     import torch

```

```

4
5     embedder = _get_embedder() # Lazy-load FashionSigLIP model
6
7     # Convert bytes to PIL Image
8     img = Image.open(io.BytesIO(raw)).convert("RGB")
9
10    # Preprocess (resize, normalize)
11    img_tensor = embedder.preprocess_val(img).unsqueeze(0).to(
embedder.device)
12
13    # Forward pass with autocast for efficiency
14    with torch.no_grad(), torch.amp.autocast(device_type=embedder.
device):
15        features = embedder.model.encode_image(img_tensor)
16        features /= features.norm(dim=-1, keepdim=True) # L2
normalization
17
18    return features.squeeze().cpu().tolist()

```

Listing 6.2: Query Image Encoding — search/image_query_engine.py

Explanation:

- Model reuse — FashionSigLIP (Marqo ViT-B-16-SigLIP) is the same embedding model used for INDEX construction, ensuring consistency
- L2 normalization — vectors normalized to unit length for cosine similarity computation
- Autocast — mixed precision (float16 where safe) reduces memory and speeds inference
- Output — standard Python list of floats, compatible with Qdrant API

Stage 3: Vector Search and Ranking

```

1 def search_by_image_bytes(raw: bytes, top_k: int = 6) -> list[dict
]:
2     """Return visually similar products for a query image."""
3     from qdrant_client import QdrantClient
4
5     # Step 1: Encode query image to vector
6     query_vector = _encode_query_image(raw)
7
8     # Step 2: Connect to Qdrant

```

```

9     client = QdrantClient(
10         host=QDRANT_HOST,
11         port=QDRANT_PORT,
12         api_key=QDRANT_API_KEY,
13         timeout=30,
14         https=False,
15         prefer_grpc=False,
16     )
17
18     # Step 3: Query nearest neighbors in "image" vector space
19     hits = client.query_points(
20         collection_name="fashion_products",
21         query=query_vector,
22         using="image",          # Named vector space (vs. text
embeddings)
23         limit=top_k,
24         with_payload=True,
25     ).points
26
27     # Step 4: Format and return results
28     products: list[dict] = []
29     for hit in hits:
30         payload = hit.payload or {}
31         products.append({
32             "image_id": str(hit.id),
33             "image_path": payload.get("image_path", ""),
34             "label": payload.get("label", ""),
35             "color": payload.get("color", ""),
36             "caption": payload.get("caption", ""),
37             "score": float(hit.score), # Cosine similarity: [0,
1], higher = more similar
38         })
39
40     return products

```

Listing 6.3: Qdrant Vector Search — search/image_query_engine.py

Explanation:

- Named vector spaces — Qdrant supports multiple embedding spaces per collection; “image” refers to pre-computed image embeddings, distinct from potential text embeddings (PATH 3)
- Cosine similarity scoring — Qdrant returns similarity scores [0, 1]; scores ≥ 0.75

typically indicate strong visual matches

- Payload retrieval — metadata (label, color, caption, image_path) attached to each vector for rich result context

6.2.3 REST API Endpoint

```
1 @app.post("/api/path2/image-search", response_model=
    Path2ImageSearchResponse)
2 async def path2_image_search_endpoint(
3     image: UploadFile = File(...),
4     session_id: str = Form(""),
5     top_k: int = Form(PATH2_DEFAULT_TOP_K),
6 ):
7     """PATH 2 image-to-image search endpoint (isolated from PATH 1
    chat/search flow)."""
8     if not _is_path2_enabled():
9         raise HTTPException(status_code=503, detail="PATH 2 image
    search is disabled")
10
11     if top_k < 1 or top_k > PATH2_MAX_TOP_K:
12         raise HTTPException(status_code=400, detail=f"top_k must be
    between 1 and {PATH2_MAX_TOP_K}")
13
14     # Validate file
15     raw = await image.read()
16     _validate_path2_png_upload(image, raw)
17
18     try:
19         # Run search
20         products = _run_path2_image_search(raw, top_k=top_k)
21
22         # Return response
23         return Path2ImageSearchResponse(
24             mode="path2",
25             session_id=session_id,
26             products=products,
27             count=len(products),
28         )
29     except HTTPException:
30         raise
31     except Exception as exc:
32         raise HTTPException(status_code=500, detail=str(exc))
```

Listing 6.4: FastAPI PATH 2 Endpoint — api/main.py

API Contract:

- Input: Multipart form with **image** (binary), optional **session_id**, optional **top_k** (1–20)
- Output: JSON with **products** (list of 6 matching items), **count**, **mode="path2"**
- Error codes: 400 (invalid file), 413 (too large), 503 (disabled), 500 (server error)

6.3 Comparison: PATH 1 vs. PATH 2

Aspect	PATH 1 (Text-to-Image)	PATH 2 (Image-to-Image)
Input	Natural language (text)	Visual file (PNG/JPG)
Parsing	Intent classification, query expansion	File validation only
Encoding	Text tokens → 512-d vector	Image pixels → 512-d vector
Search Method	BM25 (keyword) + Vector (semantic) + Fusion	Pure vector (cosine similarity only)
Reranking	BGE Reranker v2-m3 applied	No reranking (direct top-K)
Speed	500ms–2s (multi-stage)	100ms (single stage)
Quality	Good for categorical queries	Excellent for visual discovery
Use Case	“Find red shirt for men”	“Show me something like this”
Isolation	Shared with PATH 1 agent loop	Fully isolated endpoint

Table 6.1: Technical comparison between PATH 1 and PATH 2

6.4 Performance Characteristics

6.4.1 Latency Breakdown

For a typical PATH 2 query (upload 2MB image, request top-6 results):

Stage	Time (ms)	Percentage
File upload (network)	50–150	30%
Validation (PIL check)	10–30	5%
Image encoding (FashionSigLIP)	40–80	50%
Vector search (Qdrant)	10–20	10%
Result formatting	5–10	5%
Total	115–290	100%

Table 6.2: PATH 2 latency profile (P50 to P95)

Key observations:

- Bottleneck: Image encoding (FashionSigLIP forward pass) dominates runtime
- Network: Upload speed depends on user connection; typically <200ms in real deployment
- Search: Qdrant query is negligible (<20ms for 12K-item index)
- P95 target: Keep total latency <500ms for responsive UX

6.4.2 Throughput and Scaling

With a single FastAPI worker and one shared FashionSigLIP model instance:

- Single request: 150ms end-to-end
- Concurrent requests (10): 1.5s (encoder is CPU/GPU bottleneck; requests queue)
- Recommendation: Deploy 2–3 FastAPI worker processes; consider GPU acceleration for scale

6.5 Integration with Flutter Frontend

The Flutter web UI integrates PATH 2 through a dedicated image upload button:

```

1 class ChatScreen extends ConsumerWidget {
2   Future<void> _pickAndSearchImage(BuildContext context, WidgetRef
   ref) async {
3     // File picker dialog
4     final result = await FilePicker.platform.pickFiles(
5       type: FileType.image,
6       allowedExtensions: ['png', 'jpg', 'jpeg'],
7     );
8
9     if (result != null && result.files.isNotEmpty) {
10      final file = result.files.first;
11      final bytes = file.bytes;
12
13      // Show warning if file is large
14      if (bytes != null && bytes.length > 3 * 1024 * 1024) {
15        _warnIfLargeImage(context, bytes.length);
16      }
17
18      // Call chat provider (which triggers API)
19      await ref.read(chatProvider.notifier).searchByImage(bytes);
20    }
21  }
22 }

```

Listing 6.5: Flutter Image Search Integration — chat_screen.dart

Flutter workflow:

1. User clicks image icon next to send button
2. File picker opens (filters .png, .jpg, .jpeg)
3. User selects image; bytes loaded into memory
4. If size >3MB, friendly warning displayed (but upload proceeds)
5. Chat provider sends POST to `/api/path2/image-search`
6. Results displayed in chat bubbles with product thumbnails
7. Uploaded image also rendered inline in user message bubble

6.5.1 Image Rendering in Chat Bubbles

The chat UI now renders uploaded images as thumbnails (200x200px) directly in the user message bubble:

```

1 class _UserBubble extends StatelessWidget {
2   final ChatMessage message;
3
4   @override
5   Widget build(BuildContext context) {
6     return Align(
7       alignment: Alignment.centerRight,
8       child: Container(
9         constraints: BoxConstraints(maxWidth: 300),
10        padding: EdgeInsets.all(12),
11        decoration: BoxDecoration(
12          color: Colors.blue.shade100,
13          borderRadius: BorderRadius.circular(12),
14        ),
15        child: Column(
16          crossAxisAlignment: CrossAxisAlignment.end,
17          children: [
18            // Text content
19            if (message.text.isNotEmpty)
20              Text(message.text, style: TextStyle(fontSize: 14)),
21
22            // Image thumbnail (if present)
23            if (message.imageBytes != null)
24              Padding(
25                padding: EdgeInsets.only(top: 8),
26                child: ClipRRect(
27                  borderRadius: BorderRadius.circular(8),
28                  child: Image.memory(
29                    message.imageBytes!,
30                    width: 200,
31                    height: 200,
32                    fit: BoxFit.cover,
33                    errorBuilder: (context, error, stackTrace) {
34                      return Container(
35                        width: 200,
36                        height: 200,
37                        color: Colors.grey.shade300,
38                        child: Column(
39                          mainAxisAlignment: MainAxisAlignment.
40
41                          center,
42
43                          children: [
44                            Icon(Icons.broken_image),

```



```

42         SizedBox(height: 8),
43         Text('Failed to load image'),
44       ],
45     ),
46   );
47 },
48 ),
49 ),
50 ),
51 ],
52 ),
53 ),
54 );
55 }
56 }

```

Listing 6.6: Image Display in Chat Bubbles — chat_bubble.dart

UX features:

- Inline display: Images shown immediately in chat as user sends them
- Size control: Fixed 200x200px thumbnail for clean layout
- Error handling: Graceful fallback if image bytes corrupt (shows icon + message)
- Context preservation: Image stored with ChatMessage for session history

6.6 Architectural Isolation

PATH 2 is intentionally decoupled from PATH 1 to enable independent evolution:

Component	PATH 1	PATH 2
Endpoint	/api/chat	/api/path2/image-search
Search module	search/search_engine.py	search/image_query_engine.py
Agent loop	Yes (ReAct, intent, clarification)	No (direct retrieval)
LLM synthesis	Yes (Gemini summary)	No (raw product list)
Reranking	Yes (BGE)	No (direct Qdrant results)
Feature flag	Shared ENABLE_PATH2_IMAGE_SEARCH	Same flag controls both endpoints
Test suite	tests/test_search.py	tests/test_image_search.py

Table 6.3: Architectural isolation between PATH 1 and PATH 2

Benefits of isolation:

- No PATH 1 regression risk — changes to PATH 2 image search do not affect text search
- Simpler logic — PATH 2 skips intent classification, clarification, reranking (unnecessary for image search)
- Independent scaling — can cache PATH 2 results or add dedicated GPU without affecting PATH 1 agent
- Future flexibility — easy to add PATH 2 into PATH 1 loop later (e.g., multi-modal fusion in PATH 3)

6.7 Test Coverage for PATH 2

```

1 def test_encode_image_produces_normalized_vector():
2     """Verify FashionSigLIP encodes images to L2-normalized 512-d
3     vectors."""
4     from search.image_query_engine import _encode_query_image
5
6     # Load sample image
7     img_path = Path("tests/fixtures/sample_shirt.png")
8     raw = img_path.read_bytes()
9
10    # Encode
11    vector = _encode_query_image(raw)
12
13    # Assert properties
14    assert len(vector) == 512, "FashionSigLIP output should be 512-
15    dimensional"
16
17    # Check L2 norm is ~1.0
18    norm = sum(x**2 for x in vector) ** 0.5
19    assert abs(norm - 1.0) < 0.01, f"Vector should be L2-normalized
20    , got norm={norm}"
21
22 def test_search_returns_top_6_by_default():
23     """Verify image search returns top-6 products."""
24     from search.image_query_engine import search_by_image_bytes
25
26     img_path = Path("tests/fixtures/sample_shirt.png")
27     raw = img_path.read_bytes()
28
29     results = search_by_image_bytes(raw, top_k=6)

```

```

27
28     assert len(results) == 6, "Should return exactly 6 results"
29     assert all("score" in r for r in results), "Each result must
have a score"
30     assert all(0 <= r["score"] <= 1 for r in results), "Scores must
be in [0, 1]"
31
32 def test_endpoint_rejects_oversized_images():
33     """Verify API rejects images exceeding 10MB limit."""
34     # Create mock >10MB file
35     large_bytes = b'x' * (11 * 1024 * 1024) # 11 MB
36
37     response = client.post(
38         "/api/path2/image-search",
39         files={"image": ("test.png", large_bytes)}
40     )
41
42     assert response.status_code == 413, "Should return 413 for
oversized file"
43
44 def test_endpoint_accepts_valid_png():
45     """Verify API accepts valid PNG under size limit."""
46     img_path = Path("tests/fixtures/sample_shirt.png")
47
48     response = client.post(
49         "/api/path2/image-search",
50         files={"image": ("sample_shirt.png", img_path.read_bytes())
51     }
52     )
53
54     assert response.status_code == 200
55     data = response.json()
56     assert data["mode"] == "path2"
57     assert len(data["products"]) > 0

```

Listing 6.7: PATH 2 Test Suite — tests/test_image_search.py

Test coverage summary:

- Encoding: Vector dimensionality (512), normalization (L2 norm ≈ 1.0)
- Search: Top-K retrieval, score ranges, result formatting
- Validation: File size limits, unsupported formats, corrupted images

- API: HTTP status codes (200, 400, 413, 503), response schema

All 4 tests passing: Encoding, Search, File validation, API integration.

Chapter 7

End-to-End Working Example

7.1 Scenario: Finding an Outfit for a Company Meeting

7.1.1 Turn 1 — Initial Query

Turn 1 Walkthrough

User: “I want to find an outfit for a company meeting”

[Step 1 — Intent Classification]

- Classified intent: `outfit_request`
- Confidence: $0.72 \geq 0.6 \rightarrow$ sufficient to proceed
- Extracted filters: `{occasion: “office”}`

[Step 2 — Memory Lookup (PostgreSQL)]

- New session $\rightarrow$ no stored preferences

[Step 3 — Orchestrator Planning]

Gemini creates plan: `[{'tool': 'outfit_hints', 'occasion': 'office'}, {'tool': 'search', 'query': 'formal shirt office meeting'}]`

[ReAct Loop — Iterations 1–2]

Action: `outfit_hints(occasion='office')` $\rightarrow$ categories = [Shirt, Pants, Shoes]

Action: `search('formal shirt office meeting')` $\rightarrow$ 8 results after RRF + reranker

[LLM Synthesis — Gemini]

Agent responds: “For a company meeting, I suggest a smart-casual outfit: (1) A slim-fit cotton dress shirt — elegant and professional; (2) Grey or navy chino trousers; (3) Clean leather or loafer shoes.”

7.1.2 Turn 2 — Follow-up with Additional Details

Turn 2 Walkthrough

User: “Give me a white shirt and navy trousers”

[Step 1]: Intent: `follow_up`, Confidence: 0.88

Filters: {category: “Shirt”, color: “White”}

[Step 2]: Memory Agent loads session context: office occasion

[ReAct Loop]:

BM25 exact match: “Shirt White” → highest rank

FashionSigLIP semantic match: “slim fit woven cotton formal shirt”

[Result]:

Agent responds + 6-image gallery: “Here are my suggestions: (1) White slim-fit cotton shirt [SP1, SP2, SP3] (2) Navy blue chino trousers [SP4, SP5]. The white + navy combination is a classic for corporate environments.”

7.1.3 Refusal Scenario — Product Not in Database

Handling No Results Found

User: “Find me a Gucci shirt”

[Intent]: `text_search`, confidence 0.91

[ReAct Loop]: search “Gucci luxury fashion” → 0 results with relevance ≥ 0.3

[Fallback]: Lower threshold → still not sufficient

Agent politely refuses:

“Sorry, the current collection does not include Gucci brand products. However, I can find items with a similar style (luxury, minimalist, formal). Would you like me to try that instead?”

Chapter 8

Interface and Deployment

8.1 Gradio Interface

```
1 def create_gradio_app(app):
2     """Create Gradio UI integrated with FastAPI."""
3     with gr.Blocks(
4         theme=gr.themes.Monochrome(),
5         title="Fashion Agent v2.0"
6     ) as demo:
7         gr.Markdown("## Fashion Agent v2.0")
8         session_state = gr.State({"session_id": None})
9         chatbot = gr.Chatbot(label="Fashion Assistant",
10                               height=500)
11
12         with gr.Row():
13             msg = gr.Textbox(
14                 placeholder="Find a blue shirt for a meeting...",
15                 label="Your question", scale=5)
16             send = gr.Button("Send", variant="primary")
17
18         gallery = gr.Gallery(label="Products found",
19                               columns=3, height=300)
20         styling_box = gr.Textbox(label="Outfit suggestions")
21
22         gr.Examples(inputs=msg, examples=[
23             ["Find an outfit for a company meeting"],
24             ["White shirt, navy trousers"],
25             ["red dress suitable for party"],
26         ])
27     return demo
```

Listing 8.1: Gradio Chat Interface — api/main.py

8.2 Docker Deployment Configuration

```
1 # docker-compose.yml
2 services:
3     postgres:          # PostgreSQL 16 Alpine (:5432)
```

```

4     image: postgres:16-alpine
5     environment:
6         POSTGRES_DB: fashion_rag
7         POSTGRES_USER: fashion_user
8
9     qdrant:          # Qdrant Vector DB (:6333)
10        image: qdrant/qdrant:latest
11
12    fashion-api:     # FastAPI + Gradio (:8000)
13        build: .
14        environment:
15            PGHOST: postgres
16            QDRANT_HOST: qdrant
17            GEMINI_API_KEY: ${GEMINI_API_KEY}
18        volumes:
19            - ./models:/app/models    # HuggingFace cache
20            - ./images:/app/images:ro # Product images
21        depends_on:
22            postgres: {condition: service_healthy}
23            qdrant:    {condition: service_healthy}
24
25    cloudflared:     # Cloudflare Tunnel (public URL)
26        image: cloudflare/cloudflared:latest
27        command: tunnel run

```

Listing 8.2: Docker Compose — 4 services

```

1 # PostgreSQL
2 PGDATABASE=fashion_rag
3 PGUSER=fashion_user
4 PG_PASSWORD=your_secure_password
5
6 # API Keys
7 GEMINI_API_KEY=your_gemini_api_key
8 QDRANT_API_KEY=optional_qdrant_key
9 CF_TUNNEL_TOKEN=cloudflare_tunnel_token
10
11 # Paths
12 DATASET_IMAGES_HOST_PATH=./images

```

Listing 8.3: Environment variables (.env)

Chapter 9

Evaluation and Conclusion

9.1 Evaluation Metrics

Metric	Meaning	RAG v1.0	Agent v2.0
Recall@5 (PATH 1)	Top-5 correct category/color	$\geq 85\%$	$\geq 88\%$
MRR	Position of first correct result	≥ 0.75	≥ 0.80
Hit Rate@5	≥ 1 correct result in top-5	$\geq 90\%$	$\geq 93\%$
Task Completion	No crash, returns results	N/A	$\geq 95\%$
Faithfulness	No hallucinated information	N/A	≥ 0.87
Latency P95	Total ReAct loop time	<15s	<15s

Table 9.1: System evaluation metrics comparing RAG v1.0 and Agent v2.0

9.2 Data Collection and Evidence Integrity

To make the final conclusions auditable, this thesis reports not only model metrics but also the provenance of collected data, aggregation boundaries, and data-quality checks. The evidence pipeline is based on operational PostgreSQL logs generated by the deployed system.

9.2.1 Canonical Data Sources

Evidence Signal	Source Table / View	Purpose in Evaluation
LLM token usage	<code>llm_token_usage</code> , <code>session_token_summary</code> , <code>mode_cost_summary</code>	Measure input/output token consumption per session and estimate operational cost by orchestration mode.
Behavior funnel	<code>product_impressions</code> , <code>product_clicks</code> , <code>selected_items</code> , <code>product_intents</code> , <code>user_orders</code> , <code>session_path_funnel_summary</code>	Quantify user progression from impression to selection (cart add), purchase intent, and order conversion.
Retrieval quality	Evaluation query set + top-k ranked outputs	Compute Recall@5, MRR, Hit Rate@5, and faithfulness-based quality indicators.

Table 9.2: Canonical evidence sources used for thesis conclusions

9.2.2 Metric Definitions and Cohort Boundaries

Token metrics. For a cohort of sessions $\mathcal{S}$ with valid synthesis records (`call_name='synthesis'`), token usage is aggregated as:

$$T_{in} = \sum_{s \in \mathcal{S}} \text{input_tokens}_s, \quad T_{out} = \sum_{s \in \mathcal{S}} \text{output_tokens}_s, \quad T_{total} = T_{in} + T_{out} \quad (9.1)$$

When orchestration is enabled, orchestrator token fields are reported separately through `mode_cost_summary` to prevent hidden cost inflation.

Add-to-cart and behavior metrics. In this system, a cart-add event corresponds to a row in `selected_items`. Let I = impressions, C = clicks, A = cart-add selections, W = “will_buy” intent, O = orders:

$$\text{CartAddRate} = \frac{A}{I}, \quad \text{WillBuyRate} = \frac{W}{A}, \quad \text{ConversionRate} = \frac{O}{|\mathcal{S}|} \quad (9.2)$$

These definitions align with analytics endpoints in `api/analytics.py` and the funnel view in `agent/memory.py`.

Accuracy metrics. Retrieval quality is reported using Recall@5, MRR, and Hit Rate@5 (Table 9.3), computed on an evaluation cohort of N_{query} labeled queries. Every reported score must include N_{query} to avoid over-interpreting small samples.

Metric	Definition	Report with
Recall@5	Proportion of queries where at least one relevant item appears in top-5 results.	N_{query}
MRR	Mean reciprocal rank of the first relevant retrieved item.	N_{query}
Hit Rate@5	Fraction of queries with ≥ 1 relevant result in top-5.	N_{query}

Table 9.3: Accuracy reporting template with mandatory sample-size context

9.2.3 Inclusion, Exclusion, and Time-Window Rules

To ensure reproducibility, all aggregates in this chapter SHALL follow the same boundary policy:

1. Include only sessions inside the declared collection window $[t_{start}, t_{end}]$.
2. Exclude sessions with missing key fields for the target metric (e.g., missing token fields for token analysis).
3. Exclude duplicate interaction rows when the same session-event key appears multiple times due to retries.
4. Report both the original cohort size and the post-filter size to make exclusion impact explicit.

9.2.4 Data Quality and Validity Checks

Check	Validation Rule	Interpretation if Failed
Missingness	Required fields are non-null (session_id, event timestamp, metric-specific columns).	Report as undercount risk; do not claim strong support from affected metric.
Consistency	Event semantics are stable (e.g., cart-add is always <code>selected_items</code>).	Re-map metric definitions before comparing across sections.
Duplicate events	No duplicated session-event records after deduplication policy.	Treat inflated rates as invalid until corrected.
Sample balance	Cohort not dominated by one subgroup/mode.	Downgrade claim strength to “partially supported” or “inconclusive.”

Table 9.4: Validity checks for deciding whether collected data is trustworthy

This validity layer is the decision gate for “data right or wrong”: metrics are considered trustworthy only when they pass the checks in Table 9.4; otherwise, conclusions must be weakened accordingly.

9.3 Design Strengths

1. Intent-first: The agent classifies intent before acting — prevents incorrect searches.
2. Clarification gate: For ambiguous queries, asks instead of guessing — improves quality.
3. Graceful degradation: Politely refuses when no product is found, with alternative suggestions.
4. Hybrid retrieval: BM25 + FashionSigLIP + RRF leverages the strengths of both approaches.
5. PostgreSQL: Persistent, scalable, supports concurrent access better than CSV.
6. Semantic token (caption): LLM-generated descriptions enable deeper embedding understanding beyond short labels.
7. Query Expansion: Gemini-powered synonym queries increase recall for short queries.

8. Cross-encoder Reranker: BGE v2-m3 reranks results for higher precision.

9.4 Risks and Mitigation

Risk	Severity	Mitigation
High latency (multiple Gemini calls)	High	Parallel tool calls, cached expansion
LLM hallucination	High	RAGAS faithfulness ≥ 0.87 , fallback templates
BM25 rebuild on restart	Low	Serialize BM25 index to disk
PostgreSQL cold start	Low	Connection pooling
Gemini API rate limit	Medium	Batch processing, exponential backoff

Table 9.5: Risk matrix and corresponding mitigation strategies

9.5 Conclusion

This project successfully built an intelligent multimodal fashion search system following a clear two-phase roadmap. The RAG v1.0 phase established a hybrid search foundation with FashionCLIP 2.0 and BM25, while Agent v2.0 PATH 1 upgraded to a proactive architecture featuring a ReAct loop, intent classification, a clarification gate, and persistent memory via PostgreSQL.

Version v2.0 represents significant improvements over v1.0:

- Embedding: Upgraded from FashionCLIP (512-d) to Marqo-FashionSigLIP (768-d).
- Query Expansion: Added Gemini-powered synonym expansion for short queries.
- Reranker: Integrated BGE Reranker v2-m3 cross-encoder for improved precision.
- Deployment: Docker Compose stack with 4 services (PostgreSQL + Qdrant + FastAPI + Cloudflare Tunnel).

9.5.1 Evidence-Based Claim Mapping for Final Conclusion

Final claims are mapped to evidence strength so that conclusions remain proportional to verified data:

Claim	Primary Evidence	Status
Agent v2.0 improves retrieval quality over v1.0.	Recall@5, MRR, Hit Rate@5 (Section 9.2 and Section 9.2.2).	Supported
Agent behavior is robust in multi-turn interaction.	Task completion, faithfulness, and clarification behavior logs.	Supported
Operational LLM cost is explainable and monitorable.	llm_token_usage, session_token_summary, mode cost aggregation.	Partially supported
User purchase tendency is improved by recommendations.	CartAddRate / WillBuyRate / ConversionRate from behavior funnel.	Inconclusive (depends on cohort size and balance)

Table 9.6: Claim-to-evidence mapping used in the final thesis conclusion

Therefore, the final conclusion follows three rules: (1) only claims marked Supported are stated as confirmed findings, (2) Partially supported claims are reported with operational caveats, and (3) Inconclusive claims are retained as hypotheses for future data collection.

Current evidence limitations.

- Token-to-cost estimates depend on fixed public list prices and may vary over time.
- Behavior metrics are sensitive to sample imbalance across orchestration modes and user groups.
- Conversion-style claims require larger longitudinal cohorts than early-stage thesis collection windows.

Future development directions:

- Phase 3: Scale dataset to 15K–100K products, fine-tune the reranker, Redis session cache.
- Phase 4: Virtual Try-on (IDM-VTON), fine-tune FashionSigLIP on proprietary data, A/B testing.
- PATH 2: Image-to-Image search with Composed Search.

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [3] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)* (pp. 8748–8763).
- [4] Zhai, X., Mustafa, B., Kolesnikov, A., & Beyer, L. (2023). Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 11975–11986).
- [5] Marqo. (2024). Marqo-FashionSigLIP: A SigLIP-based fashion embedding model [Computer software]. Hugging Face. <https://huggingface.co/Marqo/marqo-fashionSigLIP>
- [6] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [7] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759).
- [8] Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 641–649).
- [9] Google DeepMind. (2024). Gemini 2.5 Flash: Multimodal large language model [Technical report]. Google. <https://ai.google.dev/gemini-api/docs>
- [10] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Confer-*

ence of the European Chapter of the Association for Computational Linguistics: System Demonstrations (pp. 150–158).

- [11] Qdrant. (2024). Qdrant: Vector similarity search engine [Documentation]. <https://qdrant.tech/documentation/>
- [12] The PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
- [13] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv preprint arXiv:1906.02569.
- [14] Kaggle Community. (2023). Clothing dataset full [Data set]. Kaggle. <https://www.kaggle.com/datasets/agrigorev/clothing-dataset-full>

Appendix A

Supplementary Equations and Derivations

This appendix contains supplementary mathematical material referenced in the main text.

A.1 Cosine Similarity from L2-Normalized Vectors

For two vectors $u, v \in \mathbb{R}^d$, the cosine similarity is:

$$\cos(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2} \quad (\text{A.1})$$

When both vectors are L2-normalized (i.e. $\|u\|_2 = \|v\|_2 = 1$), Equation (A.1) reduces to the dot product:

$$\cos(\hat{u}, \hat{v}) = \hat{u} \cdot \hat{v} \quad (\text{A.2})$$

This is why **FashionEmbedder** normalizes embeddings before storing them in Qdrant — it allows the use of fast dot-product search rather than full cosine computation.

A.2 BM25 Scoring Function

The BM25 score of a document d for a query $q = \{q_1, q_2, \dots, q_n\}$ is:

$$\text{BM25}(d, q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (\text{A.3})$$

where $f(q_i, d)$ is the term frequency of q_i in d , $|d|$ is the document length, avgdl is the average document length in the corpus, and k_1, b are free parameters (typically $k_1 \in [1.2, 2.0]$ and $b = 0.75$).

Appendix B

Sample Configuration Files

B.1 PostgreSQL Initialization Script

This appendix contains a sample SQL initialization script that may be used to bootstrap a fresh PostgreSQL instance for the Fashion Agent. See Table B.1 for the required environment variables.

Variable	Description
PGHOST	PostgreSQL host (e.g., <code>postgres</code> in Docker network)
PGPORT	PostgreSQL port (default: 5432)
PGDATABASE	Database name (<code>fashion_rag</code>)
PGUSER	Database user (<code>fashion_user</code>)
PGPASSWORD	Database password
GEMINI_API_KEY	Google Gemini API key
QDRANT_HOST	Qdrant host (e.g., <code>qdrant</code>)

Table B.1: Required environment variables for deployment